

Laboratorio I

a.a. 2025/2026

Prima prova in itinere
Casi strani

Leaderboard

Data e ora inizio	Data e ora consegna	Durata
28/10 12:53	28/10 13:04	0:10:42
28/10 12:31	28/10 12:44	0:13:07
28/10 14:41	28/10 14:57	0:16:04
28/10 11:58	28/10 12:22	0:24:08
27/10 11:18	27/10 11:42	0:24:09
28/10 11:15	28/10 11:49	0:33:58
02/11 23:48	03/11 00:23	0:34:56
27/10 18:05	27/10 18:45	0:39:32
02/11 15:11	02/11 15:51	0:40:11
28/10 22:14	28/10 22:57	0:42:57

Casi “strani”

Slot 1 risposta	Slot 2 risposta	Slot 3 risposta	Slot 4 risposta
<pre>function contaParole(a) { let risultato = 0; a.forEach(function(parola) { parola = parola.toLowerCase(); if (risultato[parola]) { risultato[parola]++; } else { risultato[parola] = 1; } }); return risultato; }</pre>	<pre>function dronePiuVicino(a, p, epsilon) { function distanza(p1, p2) { return Math.sqrt(Math.pow(p1.x - p2.x, 2) + Math.pow(p1.y - p2.y, 2) + Math.pow(p1.z - p2.z, 2)); } let droneVicino = a[0]; let distMin = distanza(a[0], p); for (let i = 1; i < a.length; i++) { let d = distanza(a[i], p); if (d < distMin && Math.abs(d - distMin) >= epsilon) { distMin = d; droneVicino = a[i]; } } return droneVicino; }</pre>	<pre>function filtraCorso(a, f) { let filtrati = a.filter(studente => { let [cognome, matricola] = studente; return (Number.isInteger(matricola) && matricola > 0 && matricola.toString().length === 6 && f(matricola)); }); filtrati.sort((s1, s2) => { let [cognome1, matricola1] = s1; let [cognome2, matricola2] = s2; if (cognome1 < cognome2) return -1; if (cognome1 > cognome2) return 1; return matricola2 - matricola1; }); return filtrati; }</pre>	<pre>function prodottoMigliore(a, p) { let bestName = undefined; let bestMean = -Infinity; for (const prod of a) { if (!p in prod) continue; const vals = prod[p]; if (!Array.isArray(vals) vals.length === 0) continue; let sum = 0; for (const v of vals) sum += v; const mean = sum / vals.length; if (mean > bestMean (mean === bestMean && (bestName === undefined prod.nome.localeCompare(bestName) < 0))) { bestMean = mean; bestName = prod.nome; } } return bestName; }</pre>
<pre>function contaParole(a) { let r = {}; a.forEach(function(parola) { parola = parola.toLowerCase(); if (r[parola]) { r[parola]++; } else { r[parola] = 1; } }); return r; }</pre>	<pre>function dronePiuVicino(a, p, epsilon) { function distanza(p1, p2) { return Math.sqrt(Math.pow(p1.x - p2.x, 2) + Math.pow(p1.y - p2.y, 2) + Math.pow(p1.z - p2.z, 2)); } let droneVicino = a[0]; let distMin = distanza(a[0], p); for (let i = 1; i < a.length; i++) { let d = distanza(a[i], p); if (d < distMin && Math.abs(d - distMin) >= epsilon) { distMin = d; droneVicino = a[i]; } } return droneVicino; }</pre>	<pre>function filtraCorso(a, f) { let filtrati = a.filter(studente => { let [cognome, matricola] = studente; return (Number.isInteger(matricola) && matricola > 0 && matricola.toString().length === 6 && f(matricola)); }); filtrati.sort((s1, s2) => { let [cognome1, matricola1] = s1; let [cognome2, matricola2] = s2; if (cognome1 < cognome2) return -1; if (cognome1 > cognome2) return 1; return matricola2 - matricola1; }); return filtrati; }</pre>	<pre>function prodottoMigliore(a, p) { let bestName = undefined; let bestMean = -Infinity; for (const prod of a) { if (!p in prod) continue; const vals = prod[p]; if (!Array.isArray(vals) vals.length === 0) continue; let sum = 0; for (const v of vals) sum += v; const mean = sum / vals.length; if (mean > bestMean (mean === bestMean && (bestName === undefined prod.nome.localeCompare(bestName) < 0))) { bestMean = mean; bestName = prod.nome; } } return bestName; }</pre>

Casi “strani”

Slot 1 risposta	Slot 2 risposta	Slot 3 risposta	Slot 4 risposta
<pre>function contaParole(a) { let risultato = []; a.forEach(function(parola) { parola = parola.toLowerCase(); if (risultato[parola]) { risultato[parola]++; } else { risultato[parola] = 1; } }); return risultato; }</pre>	<pre>function dronePiuVicino(a, p, epsilon) { function distanza(p1, p2) { return Math.sqrt(Math.pow(p1.x - p2.x, 2) + Math.pow(p1.y - p2.y, 2) + Math.pow(p1.z - p2.z, 2)); } let droneVicino = a[0]; let distMin = distanza(a[0], p); for (let i = 1; i < a.length; i++) { let d = distanza(a[i], p); if (d < distMin && Math.abs(d - distMin) >= epsilon) { distMin = d; droneVicino = a[i]; } } return droneVicino; }</pre>	<pre>function filtraCorso(a, f) { let filtrati = a.filter(studente => { let [cognome, matricola] = studente; return (Number.isInteger(matricola) && matricola > 0 && matricola.toString().length === 6 && f(matricola)); }); filtrati.sort((s1, s2) => { let [cognome1, matricola1] = s1; let [cognome2, matricola2] = s2; if (cognome1 < cognome2) return -1; if (cognome1 > cognome2) return 1; return matricola2 - matricola1; }); return filtrati; }</pre>	<pre>function prodottoMigliore(a, p) { let bestName = undefined; let bestMean = -Infinity; for (const prod of a) { if (!p in prod) continue; const vals = prod[p]; if (!Array.isArray(vals) vals.length === 0) continue; let sum = 0; for (const v of vals) sum += v; const mean = sum / vals.length; if (mean > bestMean (mean === bestMean && (bestName === undefined prod.nome.localeCompare(bestName) < 0))) { bestMean = mean; bestName = prod.nome; } } return bestName; }</pre>
<pre>function contaParole(a) { let risultato = []; a.forEach(function(parola) { parola = parola.toLowerCase(); if (risultato[parola]) { risultato[parola]++; } else { risultato[parola] = 1; } }); return risultato; }</pre>	<pre>function dronePiuVicino(a, p, epsilon) { function distanza(p1, p2) { return Math.sqrt(Math.pow(p1.x - p2.x, 2) + Math.pow(p1.y - p2.y, 2) + Math.pow(p1.z - p2.z, 2)); } let droneVicino = a[0]; let distMin = distanza(a[0], p); for (let i = 1; i < a.length; i++) { let d = distanza(a[i], p); if (d < distMin && Math.abs(d - distMin) >= epsilon) { distMin = d; droneVicino = a[i]; } } return droneVicino; }</pre>	<pre>function filtraCorso(a, f) { let filtrati = a.filter((cognome, matricola) => { let valida = Number.isInteger(matricola) && matricola > 0 && matricola.toString().length === 6; return valida && f(matricola); }); filtrati.sort((s1, s2) => { let [c1, m1] = s1; let [c2, m2] = s2; if (c1 < c2) return -1; if (c1 > c2) return 1; return m2 - m1; }); return filtrati; }</pre>	<pre>function filtraCorso(a, f) { let filtrati = a.filter((cognome, matricola) => { let valida = Number.isInteger(matricola) && matricola > 0 && matricola.toString().length === 6; return valida && f(matricola); }); filtrati.sort((s1, s2) => { let [c1, m1] = s1; let [c2, m2] = s2; if (c1 < c2) return -1; if (c1 > c2) return 1; return m2 - m1; }); return filtrati; }</pre>

Casi “strani”

```
function dronePiuVicino(a,p,epsilon){
  if(a != null){ //in questo if vedo se l'array è vuoto o meno
    function dis(a, p){ //dato che dovrò usare più volte la formula della radice,
      return Math.sqrt((a.x - p.x) ** 2 + (a.y - p.y) ** 2 + (a.z - p.z) ** 2)
    }
    let min = a[0] //variabile min con = a[0] così non faccio nel ciclo, una volta
    let dmin = dis(a[0], p)
    for(let i = 1; i < a.length; i++){
      let d = dis(a[i], p)
      if(d < dmin){
        if(Math.abs(dmin - d) >= epsilon){ //se maggiore o uguale a epsilon le
          min = a[i]
          dmin = d
        }
      }
    }
    return min //restituisco min
  }
}
```

```
function dronePiuVicino(a,p,epsilon){
  if(a != null){ //controllo se l'array passato è vuoto o no
    function distanza(a, p){ //creo una funzione perchè dovrò utilizzare più v
      return Math.sqrt((a.x - p.x) ** 2 + (a.y - p.y) ** 2 + (a.z - p.z) ** 2)
    }
    let minima = a[0] //inizio da a[0], così il mio ciclo non fa un passo in più
    let dminima = distanza(a[0], p) //inizializzo la distanza minima con la dist
    for(let i = 1; i < a.length; i++){
      let d2 = distanza(a[i], p)
      if(d2 < dminima){
        if(Math.abs(dminima - d2) >= epsilon){ //se la tolleranza è maggiore
          minima = a[i]
          dminima = d2
        } //invece al contrario rimane dminima e minima
      }
    }
    return minima //restituisco i punti della distanza minima da p
  }
}
```

```
function dronePiuVicino(a,p,epsilon){
  if(a != null){ //controllo se l'array passato è vuoto o no
    function distanza(a, p){ //creo una funzione perchè dovrò utilizzare più volte la formula della radice, e
      return Math.sqrt((a.x - p.x) ** 2 + (a.y - p.y) ** 2 + (a.z - p.z) ** 2)
    }
    let minima = a[0] //inizio da a[0], così il mio ciclo non fa un passo in più
    let dminima = distanza(a[0], p) //inizializzo la distanza minima con la distanza da a[0] a p
    for(let i = 1; i < a.length; i++){
      let d2 = distanza(a[i], p)
      if(d2 < dminima){
        if(Math.abs(dminima - d2) >= epsilon){ //se la tolleranza è maggiore o uguale a epsilon, allora la
          minima = a[i]
          dminima = d2
        } //invece al contrario rimane dminima e minima
      }
    }
    return minima //restituisco i punti della distanza minima da p
  }
}
```

Casi “strani”

Slot 2 risposta	Slot 3 risposta
<pre>/** Restituiamo il punto dell'array a più vicino al punto p. * In caso di parità, restituiamo il primo punto. * * @param {Array} a - Array di oggetti con chiavi x, y, z (posizione dei droni) * @param {Object} p - Oggetto con chiavi x, y, z (punto di rif.) * @param (number) epsilon - Soglia di tolleranza per considerare due distanze uguali. * @returns (Object) - il drone più vicino. */ function dronePiuVicino (a, p, epsilon) { }</pre>	<pre>/** * Filtrare e ordinare una lista di studenti in base alla loro matricola e cognome. * * @param (Array) a - Array di studenti, ogni studente è [cognome, matricola] * @param {Function} f - Funzione che prende una matricola e restituisce un boolean. * @returns {Array} - Nuovo array filtrato e ordinato. */ function filtracorso (a, f) { // filtro studenti con matricola valida che rispettano la funz. f const filtrati = a.filter (([cognome, matricola]) => { // verifico che la matricola sia un intero positivo di 6 cifre. const matricolaValida = Number.isInteger (matricola) && matricola >= 100000 && matricola <= 999999; // mantengo solo quelli che rispettano i criteri. return matricolavalida && f(matricola); }) // Ordino i risultati. // - per cognome in ordine alfabetico crescente // - a parità di cognome, per matricola in ordine decrescente filtrati.sort (((cognomeA,matricolaA), [cognomeB, matricolaB]) => { // confronto alfabetico dei cognomi const confrontocognomi = cognomeA.localeCompare(cognomeB); if (confrontocognomi !== 0) { // se i cognomi sono diversi, ritorna il risultato del confronto return confrontocognomi; } else { // se cognomi uguali, ordina per matricola decrescente return matricolaB - matricolaA } }) }</pre>

Casi “strani”

```
function prodottoMigliore(a, p) {  
  let nomeMigliore = undefined;  
  let mediaMigliore = -Infinity;  
  for (const prod of a) {  
    if (!Object.prototype.hasOwnProperty.call(prod, p)) continue;  
    const valori = prod[p];  
    if (!Array.isArray(valori) || valori.length === 0) continue;  
    const somma = valori.reduce((s, v) => s + v, 0);  
    const media = somma / valori.length;  
    if (media > mediaMigliore) {  
      mediaMigliore = media;  
      nomeMigliore = prod.nome;  
    } else if (media === mediaMigliore && typeof prod.nome === "string") {  
      if (nomeMigliore === undefined || prod.nome.localeCompare(nomeMigliore) < 0) {  
        nomeMigliore = prod.nome;  
      }  
    }  
  }  
  return nomeMigliore;  
}
```

GPT

```
function dronePiuVicino(a, p, epsilon) {
  if (!Array.isArray(a) || a.length === 0) {
    throw new Error("L'input deve essere un array non vuoto.");
  }
  let distanzaMinima = Infinity;
  let dronePiuVicino = null;
  for (let i = 0; i < a.length; i++) {
    const drone = a[i];
    if (Math.abs(distanza - distanzaMinima) < epsilon) {
    } else if (distanza < distanzaMinima) {
      distanzaMinima = distanza;
      dronePiuVicino = drone;
    }
  }
  return dronePiuVicino;
}
```

```
function dronePiuVicino(a, p, epsilon) {
  let distanzaMinima = Infinity;
  let droneVicino = null;
  for (let drone of a) {
    if (distanza < distanzaMinima) {
      distanzaMinima = distanza;
      droneVicino = drone;
    } else if (Math.abs(distanza - distanzaMinima) < epsilon) {
      continue;
    }
  }
  return droneVicino;
}
```

```
function dronePiuVicino(a, p, epsilon) {
  let distanzaMinima = Infinity;
  let indiceDistanzaMinima = -1;
  for (let i = 0; i < a.length; i++) {
    if (distanza < distanzaMinima) {
      distanzaMinima = distanza;
      indiceDistanzaMinima = i;
    } else if (Math.abs(distanza - distanzaMinima) < epsilon) {
      continue;
    }
  }
  return a[indiceDistanzaMinima];
}
```

```
function dronePiuVicino(a, p, epsilon) {
  if (a.length === 0) {
    throw new Error("L'array di punti non può essere vuoto.");
  }
  let dronePiuVicino = null;
  let distanzaMinima = Infinity;
  for (let i = 0; i < a.length; i++) {
    const drone = a[i];
    if (distanza < distanzaMinima) {
      dronePiuVicino = drone;
      distanzaMinima = distanza;
    } else if (Math.abs(distanza - distanzaMinima) < epsilon) {
    }
  }
  return dronePiuVicino;
}
```

```
function dronePiuVicino(a, p, epsilon) {
  let minDistance = Infinity;
  let closestDrone = null;
  for (let i = 0; i < a.length; i++) {
    if (distance < minDistance) {
      minDistance = distance;
      closestDrone = a[i];
    } else if (Math.abs(distance - minDistance) < epsilon) {
    }
  }
  return closestDrone;
}
```